

Windows File Permissions

Windows permissions work fundamentally differently from Linux's simple owner/group/other model. Instead, Windows uses **ACLs (Access Control Lists)** — a much more granular and complex system. Since you're building AD pentesting skills, this is essential groundwork for later topics like ACL abuse in Active Directory.

1. The Core Concept — NTFS vs. Linux Permissions

Linux	Windows (NTFS)
3 entities: owner, group, others	Unlimited entities — any user/group can get an entry
Fixed rwx bits	Dozens of granular permission types
One permission set per file	A list of permission entries (ACL)
Permissions inherited only if you set them	Inheritance is a first-class, automatic concept

Windows doesn't use "owner/group/other" — instead every file/folder has a **Security Descriptor** containing:

- **Owner** — SID of who owns the object
- **DACL** (Discretionary Access Control List) — the actual permissions list (who can/can't do what)
- **SACL** (System Access Control List) — auditing rules (what gets logged)

2. SIDs — The Foundation

Windows doesn't identify users by name internally — it uses **SIDs (Security Identifiers)**, unique strings like:

```
S-1-5-21-3623811015-3361044348-30300820-1013
```

Breaking it down:

- **S-1-5** — standard SID prefix

- `21-3623811015-3361044348-30300820` — the domain/machine identifier
- `1013` — the **RID (Relative ID)** — identifies the specific user/group

Well-known SIDs worth memorizing (they show up constantly in AD pentesting):

SID	Meaning
S-1-1-0	Everyone
S-1-5-11	Authenticated Users
S-1-5-18	SYSTEM (Local System — the "root" of Windows)
S-1-5-32-544	Administrators (local)
S-1-5-32-545	Users (local)
RID 500	Built-in Administrator account (always ends in -500)
RID 512	Domain Admins (in AD)

`whoami /user` shows your own SID. `whoami /groups` shows every group SID you belong to — this matters because Windows checks *all* your group SIDs when evaluating access, not just your username.

3. NTFS Permission Types

Unlike Linux's 3 bits, NTFS has granular permissions, shown in two levels of detail:

Basic (Standard) Permissions — what you see in the GUI

Permission	What it Allows
Full Control	Everything — read, write, execute, delete, change permissions, take ownership
Modify	Read, write, execute, delete (but can't change permissions/ownership)
Read & Execute	View + run, no editing
List Folder Contents	(folders only) — see what's inside
Read	View file contents/attributes
Write	Modify/create content

Advanced (Granular) Permissions — the real underlying bits

```
Full Control
Traverse Folder / Execute File
List Folder / Read Data
Read Attributes
Read Extended Attributes
Create Files / Write Data
Create Folders / Append Data
Write Attributes
Write Extended Attributes
Delete Subfolders and Files
Delete
Read Permissions
Change Permissions
Take Ownership
```

Each "basic" permission (like Modify) is actually just a bundled combination of several of these granular ones. This granularity is exactly why NTFS permission auditing is more complex than Linux — you can grant *just* "Take Ownership" without granting "Write," for example, which creates subtle privilege escalation paths.

4. Viewing Permissions — GUI and CLI

GUI: Right-click file/folder → Properties → Security tab → Edit/Advanced

CLI — `icacls` (the main tool you'll use):

```
icacls C:\Users\Omkar\Documents
```

Output looks like:

```
C:\Users\Omkar\Documents BUILTIN\Administrators:(F)
                        NT AUTHORITY\SYSTEM:(F)
                        OMKAR-PC\Omkar:(F)
                        BUILTIN\Users:(RX)
```

Permission abbreviations in `icacls` :

Abbreviation	Meaning
<code>F</code>	Full control
<code>M</code>	Modify
<code>RX</code>	Read & Execute

Abbreviation	Meaning
R	Read
W	Write
D	Delete
(OI)	Object Inherit — applies to files within this folder
(CI)	Container Inherit — applies to subfolders
(IO)	Inherit Only — doesn't apply to this object itself, only children
(I)	Inherited — this permission came from the parent folder

PowerShell equivalent:

```
Get-Acl C:\Users\Omkar\Documents | Format-List
```

5. Changing Permissions — **icacls**

```
:: Grant full control to a user
icacls file.txt /grant Omkar:F

:: Grant read+execute
icacls file.txt /grant Omkar:RX

:: Remove a user's permissions entirely
icacls file.txt /remove Omkar

:: Reset to inherit from parent (removes explicit permissions)
icacls file.txt /reset

:: Apply recursively to a folder and everything inside
icacls C:\Shared /grant Users:M /T

:: Grant, but only apply to subfolders/files (not the folder itself)
icacls C:\Shared /grant Users:(OI)(CI)F
```

PowerShell equivalent (more scriptable, used in AD environments):

```
$acl = Get-Acl C:\Shared
$rule = New-Object System.Security.AccessControl.FileSystem
AccessRule("0mkar","FullControl","Allow")
$acl.SetAccessRule($rule)
Set-Acl C:\Shared $acl
```

6. Ownership — **takeown**

```
takeown /f C:\protected_file.txt
```

This is a critical privilege escalation primitive: if you have the **SeTakeOwnershipPrivilege** (even without being an admin), you can take ownership of *any* file on the system — and once you own it, you can grant yourself full permissions via **icacls**, even if you previously had zero access.

```
whoami /priv
```

Check this output for privileges like **SeTakeOwnershipPrivilege**, **SeBackupPrivilege**, or **SeRestorePrivilege** — these are classic Windows privilege escalation vectors, since each one lets you bypass normal ACL checks in a specific way.

7. Inheritance

By default, permissions **cascade down** from parent folders to child files/folders. This is Windows' equivalent of "everything inherits from the directory it lives in," but far more explicit and controllable than Linux.

- **Inherited permissions** show as greyed out in the GUI (can't be directly removed, only "disabled" or overridden at the parent)
- **Explicit permissions** set directly on an object always **override** inherited ones if there's a conflict
- You can **break inheritance**: right-click → Advanced → "Disable inheritance" → choose to either convert inherited permissions to explicit ones, or remove them all and start fresh

Order of precedence when Windows evaluates access (important for AD pentesting):

1. Explicit **Deny** — always wins over everything
2. Explicit **Allow**
3. Inherited **Deny**
4. Inherited **Allow**

This is a critical difference from Linux: **Deny rules exist and always take priority**. A single explicit Deny entry can block access even if 10 other groups grant Full Control.

8. NTFS Permissions vs. Share Permissions

This trips up a lot of people, so it's worth being precise:

	NTFS Permissions	Share Permissions
Applies to	Local + network access	Network access only (SMB)
Granularity	Very detailed (14+ permission types)	Only 3: Full Control, Change, Read
Set via	Security tab / <code>icacls</code>	Sharing tab / <code>net share</code>

When both exist (accessing a file over the network via a share), Windows applies the MOST RESTRICTIVE of the two.

Example: If Share permissions grant "Everyone: Full Control" but NTFS permissions only grant "Everyone: Read" → the effective network access is **Read only**, because NTFS is more restrictive.

This is a very common **misconfiguration pentesters look for**: admins often set Share = Full Control (thinking NTFS handles the "real" security) and then misconfigure NTFS too loosely, or vice-versa, creating exploitable gaps.

```
net share                                # list shares on a machine
net share ShareName /delete             # remove a share
```

In AD environments, this is exactly what tools like **PowerView** or **SharpHound** enumerate when hunting for **writable/misconfigured SMB shares** — a very common initial foothold technique.

9. Special / Built-in Accounts Worth Knowing

Account	Role
SYSTEM	Highest local privilege — even above local Administrator in some respects. Most services run as SYSTEM.
Administrator	Local admin — full control over the local machine
Everyone	Includes all users, even anonymous/guest in older configs
Authenticated Users	All logged-in users, excludes anonymous/guest
Network Service / Local Service	Lower-privileged service accounts, used to limit blast radius if a service is compromised

Why SYSTEM matters for you specifically: in Windows privilege escalation, the entire goal is usually going from a low-priv user → local Administrator → SYSTEM. Many exploits (like certain service exploits, token impersonation attacks e.g. **Potato family exploits** — JuicyPotato, RoguePotato) specifically abuse SYSTEM-level service tokens.

10. Integrity Levels (a layer beyond ACLs)

Windows also has **Mandatory Integrity Control (MIC)** — a separate layer from DACLs, introduced in Vista, mainly relevant to **UAC (User Account Control)**.

Integrity Level	Example
System	SYSTEM processes
High	Elevated/Administrator processes (UAC-approved)
Medium	Standard user processes (default)
Low	Sandboxed processes (e.g., browser, protected mode)

A process can't write to an object with a **higher** integrity level than itself, regardless of what the DACL says — this is why even a local admin's processes normally run at Medium integrity until UAC elevation bumps them to High. This is the exact mechanism **UAC bypass techniques** target during privilege escalation.

```
icaccls file.txt :: sometimes shows a mandatory label line
like:
Mandatory Label\High Mandatory Level:(NW)
```

11. Practical Privilege Escalation Angle (Study Context)

Since this connects directly to your VAPT/AD interests:

a) Weak folder/file permissions on services

```
icacls "C:\Program Files\VulnerableApp\service.exe"
```

If a service binary is writable by a low-priv user, but the service itself runs as SYSTEM, you can replace the binary with a malicious one and restart the service (or wait for a reboot) to get SYSTEM-level code execution. This is a textbook Windows privesc technique, checked automatically by tools like **WinPEAS** and **PowerUp**.

b) AlwaysInstallElevated misconfiguration

Not file permissions exactly, but related registry-based misconfig — if set, any user can install an `.msi` with SYSTEM privileges, a very common CTF/OSCP-style finding.

c) Unquoted service paths

```
wmic service get name,pathname,startmode | findstr /i /v  
"C:\Windows"
```

If a service path has spaces and isn't quoted (e.g., `C:\Program Files\My App\service.exe`), and you have write access to `C:\`, you can drop a malicious `Program.exe` that gets executed instead — a classic finding.

d) Scheduled tasks / cron equivalent

```
schtasks /query /fo LIST /v
```

Same logic as Linux cron abuse — if a SYSTEM-run scheduled task points to a script you can write to, you inject your payload there.

e) Registry-based service permissions

```
accesschk.exe -uwcqv "Users" *
```

(Sysinternals tool) — hunts for services where a low-priv group has write access to the service's registry configuration, letting you redirect what binary it

launches.

12. Quick Comparison Table — Linux vs Windows Mental Model

Concept	Linux	Windows
Identity	UID/GID	SID
Permission model	rwx bits (owner/group/other)	ACL (DACL) with unlimited entries
Elevated execution	SUID bit	Service running as SYSTEM + writable binary
"Take what you don't own"	N/A (root only)	<code>takeown</code> + <code>SeTakeOwnershipPrivilege</code>
Deny rules	Don't exist (only grant)	Explicit Deny ACE, always wins
Special execution context	Sticky bit, SGID	Integrity Levels (Mandatory Access Control)
Enumeration tool	<code>find -perm</code> , manual <code>ls -l</code>	<code>icacls</code> , <code>accesschk</code> , WinPEAS, PowerUp

Quick Reference Cheat Sheet

```
icacls file.txt # view permissions
icacls file.txt /grant Omkar:Full # grant full control
icacls file.txt /remove Omkar # remove user's ACL entry
icacls C:\Folder /grant Users:M /T # recursive grant
takeown /f file.txt # take ownership
whoami /priv # check your privileges
whoami /groups # check your group SIDs
net share # list SMB shares
schtasks /query /fo LIST /v # list scheduled tasks
```

```
wmic service get name,pathname,startmode # find unquoted service paths
```
